

```

        z[j][k] -= (f*e[k]+g*z[i][k]);
    }
}
} else
    e[i]=z[i][1];
    d[i]=h;
}
if (yesvecs) d[0]=0.0;
e[0]=0.0;
for (i=0;i<n;i++) {
    if (yesvecs) {
        if (d[i] != 0.0) {
            for (j=0;j<i;j++) {
                g=0.0;
                for (k=0;k<i;k++)
                    g += z[i][k]*z[k][j];
                for (k=0;k<i;k++)
                    z[k][j] -= g*z[k][i];
            }
        }
        d[i]=z[i][i];
        z[i][i]=1.0;
        for (j=0;j<i;j++) z[j][i]=z[i][j]=0.0;
    } else {
        d[i]=z[i][i];
    }
}
}
}

```

Begin accumulation of transformation matrices.
This block skipped when $i=0$.

Use $\mathbf{u}$ and $\mathbf{u}/H$ stored in $\mathbf{z}$ to form $\mathbf{P}\cdot\mathbf{Q}$.

Reset row and column of $\mathbf{z}$ to identity matrix for next iteration.

Only this statement remains.

CITED REFERENCES AND FURTHER READING:

- Golub, G.H., and Van Loan, C.F. 1996, *Matrix Computations*, 3rd ed. (Baltimore: Johns Hopkins University Press), §5.1.[1]
- Smith, B.T., et al. 1976, *Matrix Eigensystem Routines — EISPACK Guide*, 2nd ed., vol. 6 of *Lecture Notes in Computer Science* (New York: Springer).
- Wilkinson, J.H., and Reinsch, C. 1971, *Linear Algebra*, vol. II of *Handbook for Automatic Computation* (New York: Springer).[2]

11.4 Eigenvalues and Eigenvectors of a Tridiagonal Matrix

We now turn to the second step in the grand strategy outlined in §11.2, namely computing the eigenvectors and eigenvalues of a tridiagonal matrix.

11.4.1 Evaluation of the Characteristic Polynomial

Once our original real symmetric matrix has been reduced to tridiagonal form, one possible way to determine its eigenvalues is to find the roots of the characteristic polynomial $p_n(\lambda)$ directly. The characteristic polynomial of a tridiagonal matrix can be evaluated for any trial value of λ by an efficient recursion relation (see [1], for example). The polynomials of lower degree produced during the recurrence form a Sturmian sequence that can be used to localize the eigenvalues to intervals on the

real axis. A root-finding method such as bisection or Newton's method can then be employed to refine the intervals. The corresponding eigenvectors can then be found by inverse iteration (see §11.8).

Procedures based on these ideas can be found in [2,3]. If, however, more than a small fraction of all the eigenvalues and eigenvectors is required, then the factorization method next considered is much more efficient.

11.4.2 The *QR* and *QL* Algorithms

The basic idea behind the *QR* algorithm is that any real matrix can be decomposed in the form

$$\mathbf{A} = \mathbf{Q} \cdot \mathbf{R} \quad (11.4.1)$$

where $\mathbf{Q}$ is orthogonal and $\mathbf{R}$ is upper triangular. For a general matrix, the decomposition is constructed by applying Householder transformations to annihilate successive columns of $\mathbf{A}$ below the diagonal (see §2.10).

Now consider the matrix formed by writing the factors in (11.4.1) in the opposite order:

$$\mathbf{A}' = \mathbf{R} \cdot \mathbf{Q} \quad (11.4.2)$$

Since $\mathbf{Q}$ is orthogonal, equation (11.4.1) gives $\mathbf{R} = \mathbf{Q}^T \cdot \mathbf{A}$. Thus equation (11.4.2) becomes

$$\mathbf{A}' = \mathbf{Q}^T \cdot \mathbf{A} \cdot \mathbf{Q} \quad (11.4.3)$$

We see that $\mathbf{A}'$ is an orthogonal transformation of $\mathbf{A}$.

You can verify that a *QR* transformation preserves the following properties of a matrix: symmetry, tridiagonal form, and Hessenberg form (to be defined in §11.6).

There is nothing special about choosing one of the factors of $\mathbf{A}$ to be upper triangular; one could equally well make it lower triangular. This is called the *QL* algorithm, since

$$\mathbf{A} = \mathbf{Q} \cdot \mathbf{L} \quad (11.4.4)$$

where $\mathbf{L}$ is lower triangular. (The standard, but confusing, nomenclature *R* and *L* stands for whether the *right* or *left* of the matrix is nonzero.)

Recall that in the Householder reduction to tridiagonal form in §11.3, we started in column $n - 1$ of the original matrix. To minimize roundoff, we then exhorted you to put the biggest elements of the matrix in the lower right-hand corner, if you can. If we now wish to diagonalize the resulting tridiagonal matrix, the *QL* algorithm will have smaller roundoff than the *QR* algorithm, so we shall use *QL* henceforth.

The *QL* algorithm consists of a *sequence* of orthogonal transformations:

$$\begin{aligned} \mathbf{A}_s &= \mathbf{Q}_s \cdot \mathbf{L}_s \\ \mathbf{A}_{s+1} &= \mathbf{L}_s \cdot \mathbf{Q}_s \quad (= \mathbf{Q}_s^T \cdot \mathbf{A}_s \cdot \mathbf{Q}_s) \end{aligned} \quad (11.4.5)$$

The following (nonobvious!) theorem is the basis of the algorithm for a general matrix $\mathbf{A}$: (i) If $\mathbf{A}$ has eigenvalues of different absolute value $|\lambda_i|$, then $\mathbf{A}_s \rightarrow$ [lower triangular form] as $s \rightarrow \infty$. The eigenvalues appear on the diagonal in increasing order of absolute magnitude. (ii) If $\mathbf{A}$ has an eigenvalue $|\lambda_i|$ of multiplicity p , $\mathbf{A}_s \rightarrow$ [lower triangular form] as $s \rightarrow \infty$, except for a diagonal block matrix of

order p , whose eigenvalues $\rightarrow \lambda_i$. The proof of this theorem is fairly lengthy; see, for example, [4].

The workload in the QL algorithm is $O(n^3)$ per iteration for a general matrix, which is prohibitive. However, the workload is only $O(n)$ per iteration for a tridiagonal matrix and $O(n^2)$ for a Hessenberg matrix, which makes it highly efficient on these forms.

In this section we are concerned only with the case where $\mathbf{A}$ is a real, symmetric, tridiagonal matrix. All the eigenvalues λ_i are thus real. According to the theorem, if any λ_i has a multiplicity p , then there must be at least $p - 1$ zeros on the sub- and superdiagonals. Thus the matrix can be split into submatrices that can be diagonalized separately, and the complication of diagonal blocks that can arise in the general case is irrelevant.

In the proof of the theorem quoted above, one finds that in general a superdiagonal element converges to zero like

$$a_{ij}^{(s)} \sim \left(\frac{\lambda_i}{\lambda_j} \right)^s \quad (11.4.6)$$

Although $\lambda_i < \lambda_j$, convergence can be slow if λ_i is close to λ_j . Convergence can be accelerated by the technique of *shifting*: If k is any constant, then $\mathbf{A} - k\mathbf{1}$ has eigenvalues $\lambda_i - k$. If we decompose

$$\mathbf{A}_s - k_s \mathbf{1} = \mathbf{Q}_s \cdot \mathbf{L}_s \quad (11.4.7)$$

so that

$$\begin{aligned} \mathbf{A}_{s+1} &= \mathbf{L}_s \cdot \mathbf{Q}_s + k_s \mathbf{1} \\ &= \mathbf{Q}_s^T \cdot \mathbf{A}_s \cdot \mathbf{Q}_s \end{aligned} \quad (11.4.8)$$

then the convergence is determined by the ratio

$$\frac{\lambda_i - k_s}{\lambda_j - k_s} \quad (11.4.9)$$

The idea is to choose the shift k_s at each stage to maximize the rate of convergence. A good choice for the shift initially would be k_s close to λ_0 , the smallest eigenvalue. Then the first row of off-diagonal elements would tend rapidly to zero. However, λ_0 is not usually known a priori. A very effective strategy in practice (although there is no proof that it is optimal) is to compute the eigenvalues of the leading 2×2 diagonal submatrix of $\mathbf{A}$. Then set k_s equal to the eigenvalue closer to a_{00} .

More generally, suppose you have already found r eigenvalues of $\mathbf{A}$. Then you can *deflate* the matrix by crossing out the first r rows and columns, leaving

$$\mathbf{A} = \begin{bmatrix} 0 & & \cdots & \cdots & & 0 \\ & \cdots & & & & \\ & & 0 & & & \\ \vdots & & & d_r & e_r & \vdots \\ \vdots & & & e_r & d_{r+1} & \\ & & & & \cdots & 0 \\ 0 & & \cdots & & 0 & d_{n-2} & e_{n-2} \\ & & & & & e_{n-2} & d_{n-1} \end{bmatrix} \quad (11.4.10)$$

which is known since $\mathbf{q}_{n-1}$ is known. Then equation (11.4.13) gives

$$\alpha_{n-1} \mathbf{q}_{n-2}^T = \mathbf{z}_{n-2}^T \quad (11.4.16)$$

where

$$\mathbf{z}_{n-2}^T \equiv \mathbf{q}_{n-1}^T \cdot \mathbf{A} - \beta_{n-1} \mathbf{q}_{n-1}^T \quad (11.4.17)$$

is known. Therefore

$$\alpha_{n-1}^2 = \mathbf{z}_{n-2}^T \mathbf{z}_{n-2}, \quad (11.4.18)$$

or

$$\alpha_{n-1} = |\mathbf{z}_{n-2}| \quad (11.4.19)$$

and

$$\mathbf{q}_{n-2}^T = \mathbf{z}_{n-2}^T / \alpha_{n-1} \quad (11.4.20)$$

(where α_{n-1} is nonzero by hypothesis). Similarly, one can show by induction that if we know $\mathbf{q}_{n-1}, \mathbf{q}_{n-2}, \dots, \mathbf{q}_{n-j}$ and the α 's, β 's, and γ 's up to level $n-j$, one can determine the quantities at level $n-(j+1)$.

To apply the lemma in practice, suppose one can somehow find a tridiagonal matrix $\bar{\mathbf{A}}_{s+1}$ such that

$$\bar{\mathbf{A}}_{s+1} = \bar{\mathbf{Q}}_s^T \cdot \bar{\mathbf{A}}_s \cdot \bar{\mathbf{Q}}_s \quad (11.4.21)$$

where $\bar{\mathbf{Q}}_s^T$ is orthogonal and has the same last row as $\mathbf{Q}_s^T$ in the original QL algorithm. Then $\bar{\mathbf{Q}}_s = \mathbf{Q}_s$ and $\bar{\mathbf{A}}_{s+1} = \mathbf{A}_{s+1}$.

Now, in the original algorithm, from equation (11.4.11) we see that the last row of $\mathbf{Q}_s^T$ is the same as the last row of $\mathbf{P}_{n-1}^{(s)}$. But recall that $\mathbf{P}_{n-1}^{(s)}$ is a plane rotation designed to annihilate the $(n-2, n-1)$ element of $\mathbf{A}_s - k_s \mathbf{1}$. A simple calculation using the expression (11.1.1) shows that it has parameters

$$c = \frac{d_{n-1} - k_s}{\sqrt{e_{n-1}^2 + (d_{n-1} - k_s)^2}}, \quad s = \frac{-e_{n-2}}{\sqrt{e_{n-1}^2 + (d_{n-1} - k_s)^2}} \quad (11.4.22)$$

The matrix $\mathbf{P}_{n-1}^{(s)} \cdot \mathbf{A}_s \cdot \mathbf{P}_{n-1}^{(s)T}$ is tridiagonal with two extra elements:

$$\begin{bmatrix} \dots & & & & \\ & \times & \times & \times & \\ & & \times & \times & \times & \mathbf{x} \\ & & & \times & \times & \times \\ & & & & \mathbf{x} & \times & \times \end{bmatrix} \quad (11.4.23)$$

We must now reduce this to tridiagonal form with an orthogonal matrix whose last row is $[0, 0, \dots, 0, 1]$ so that the last row of $\bar{\mathbf{Q}}_s^T$ will stay equal to $\mathbf{P}_{n-1}^{(s)}$. This can be done by a sequence of Householder or Givens transformations. For the special form of the matrix (11.4.23), Givens is better. We rotate in the plane $(n-3, n-2)$ to annihilate the $(n-3, n-1)$ element. [By symmetry, the $(n-1, n-3)$ element will also be zeroed.] This leaves us with a tridiagonal form except for the extra elements $(n-4, n-2)$ and $(n-2, n-4)$. We annihilate these with a rotation in the $(n-4, n-3)$ -plane, and so on. Thus a sequence of $n-2$ Givens rotations is required. The result is that

$$\mathbf{Q}_s^T = \bar{\mathbf{Q}}_s^T = \bar{\mathbf{P}}_1^{(s)} \cdot \bar{\mathbf{P}}_2^{(s)} \dots \bar{\mathbf{P}}_{n-2}^{(s)} \cdot \mathbf{P}_{n-1}^{(s)} \quad (11.4.24)$$

where the $\bar{\mathbf{P}}$'s are the Givens rotations and $\mathbf{P}_{n-1}$ is the same plane rotation as in the original algorithm. Then equation (11.4.21) gives the next iterate of $\mathbf{A}$. Note that the shift k_s enters implicitly through the parameters (11.4.22).

The following routine `tqli` (“Tridiagonal QL Implicit”), based algorithmically on the implementations in [2,3], works extremely well in practice. The number of iterations for the first few eigenvalues might be four or five, say, but meanwhile the off-diagonal elements in the lower right-hand corner have been reduced too. The later eigenvalues are liberated with very little work. The average number of iterations per eigenvalue is typically 1.3–1.6. The operation count per iteration is $O(n)$, with a fairly large effective coefficient, say $\sim 20n$. The total operation count for the diagonalization is then very roughly $\sim 20n \times (1.3\text{--}1.6)n \sim 30n^2$. If the eigenvectors are required, the statements indicated by comments are included and there is an additional, much larger, workload of about $6n^3$ operations.

eigen_sym.h

```
void Symmeig::tqli()
    QL algorithm with implicit shifts to determine the eigenvalues and (optionally) the eigenvectors
    of a real, symmetric, tridiagonal matrix, or of a real symmetric matrix previously reduced by
    tred2 (§11.3). On input, d[0..n-1] contains the diagonal elements of the tridiagonal matrix.
    On output, it returns the eigenvalues. The vector e[0..n-1] inputs the subdiagonal elements
    of the tridiagonal matrix, with e[0] arbitrary. On output e is destroyed. If the eigenvectors of
    a tridiagonal matrix are desired, the matrix z[0..n-1][0..n-1] is input as the identity matrix.
    If the eigenvectors of a matrix that has been reduced by tred2 are required, then z is input as
    the matrix output by tred2. In either case, column k of z returns the normalized eigenvector
    corresponding to d[k].
{
    Int m,l,iter,i,k;
    Doub s,r,p,g,f,dd,c,b;
    const Doub EPS=numeric_limits<Doub>::epsilon();
    for (i=1;i<n;i++) e[i-1]=e[i];           Convenient to renumber the el-
    e[n-1]=0.0;                               ements of e.
    for (l=0;l<n;l++) {
        iter=0;
        do {
            for (m=l;m<n-1;m++) {             Look for a single small subdi-
                dd=abs(d[m])+abs(d[m+1]);       agonal element to split the
                if (abs(e[m]) <= EPS*dd) break;  matrix.
            }
            if (m != l) {
                if (iter++ == 30) throw("Too many iterations in tqli");
                g=(d[l+1]-d[l])/(2.0*e[l]);      Form shift.
                r=pythag(g,1.0);
                g=d[m]-d[l]+e[l]/(g+SIGN(r,g));  This is  $d_m - k_s$ .
                s=c=1.0;
                p=0.0;
                for (i=m-1;i>=l;i--) {          A plane rotation as in the origi-
                    f=s*e[i];                    nal  $QL$ , followed by Givens
                    b=c*e[i];                    rotations to restore tridiag-
                    e[i+1]=(r=pythag(f,g));       onal form.
                    if (r == 0.0) {               Recover from underflow.
                        d[i+1] -= p;
                        e[m]=0.0;
                        break;
                    }
                    s=f/r;
                    c=g/r;
                    g=d[i+1]-p;
                    r=(d[i]-g)*s+2.0*c*b;
                    d[i+1]=g+(p=s*r);
                    g=c*r-b;
                    if (yesvecs) {
                        for (k=0;k<n;k++) {      Form eigenvectors.
                            f=z[k][i+1];
                            z[k][i+1]=s*z[k][i]+c*f;
                        }
                    }
                }
            }
        } while (abs(e[m]) > EPS*dd);
    }
}
```

```

        z[k][i]=c*z[k][i]-s*f;
    }
}
}
if (r == 0.0 && i >= 1) continue;
d[1] -= p;
e[1]=g;
e[m]=0.0;
}
} while (m != 1);
}
}

Doub Symmeig::pythag(const Doub a, const Doub b) {
Computes  $(a^2 + b^2)^{1/2}$  without destructive underflow or overflow.
    Doub absa=abs(a), absb=abs(b);
    return (absa > absb ? absa*sqrt(1.0+SQR(absb/absa)) :
        (absb == 0.0 ? 0.0 : absb*sqrt(1.0+SQR(absa/absb))));
}

```

11.4.4 Newer Methods

There are two newer algorithms for tridiagonal symmetric systems that are generally more efficient than the *QL* method, especially for large matrices. The first is the *divide-and-conquer method* [5]. This method divides the tridiagonal matrix into two halves, solves the eigenproblems in each of the two halves, and then stitches the two solutions together to generate the solution of the original problem. The method is applied recursively, with the *QL* method used once the matrices are sufficiently small. The method is implemented in LAPACK as *dstevd* and is about 2.5 times faster than the *QL* method for large matrices.

The fastest method of all for the vast majority of matrices is the *MRRR algorithm* (Multiple Relatively Robust Representations) [6]. As we will see in §11.8, inverse iteration can determine the eigenvectors of a tridiagonal matrix in $O(n^2)$ operations. However, clustered eigenvalues lead to eigenvectors that are not properly orthogonal to one another. Using a procedure like Gram-Schmidt to orthogonalize the vectors is $O(n^3)$. The MRRR algorithm is a sophisticated version of inverse iteration that is $O(n^2)$ without requiring Gram-Schmidt. An implementation is available in LAPACK as *dstegr*.

CITED REFERENCES AND FURTHER READING:

- Acton, F.S. 1970, *Numerical Methods That Work*, 1990, corrected edition (Washington, DC: Mathematical Association of America), pp. 331–335.[1]
- Wilkinson, J.H., and Reinsch, C. 1971, *Linear Algebra*, vol. II of *Handbook for Automatic Computation* (New York: Springer).[2]
- Smith, B.T., et al. 1976, *Matrix Eigensystem Routines — EISPACK Guide*, 2nd ed., vol. 6 of *Lecture Notes in Computer Science* (New York: Springer).[3]
- Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), §6.6.4.[4]
- Cuppen, J.J.M. 1981, “A Divide-and-Conquer Method for the Symmetric Tridiagonal Eigenproblem,” *Numerische Mathematik*, vol. 36, pp. 177–195.[5]
- Dhillon, I.S., and Parlett, B.N. 2004, “Multiple Representations to Compute Orthogonal Eigenvectors of Symmetric Tridiagonal Matrices,” *Linear Algebra and Its Applications*, vol. 387, pp. 1–28.[6]